

DASE7506 MP1: A Small Language Model for WikiText-2

Student ID: 3036708440

Implementation: `student`

Final configuration: width 224, 7 attention heads, 6 Transformer blocks, context 256, GELU MLP, LayerNorm, 6,000 updates, EMA decay 0.999, cosine learning-rate floor 0.99, seed 18.

Abstract

This report describes a small causal language model trained from scratch on the supplied WikiText-2 training text. The work followed the fixed BPE-2048 tokenizer, independent causal windows of 256 tokens, and the supplied FP32 evaluator. Development used validation BPB; the test split was kept for the frozen final evaluation.

The main improvement came from increasing model capacity from the supplied 1.09M-parameter baseline to a 4.15M-parameter model and from using a longer, more carefully selected training recipe. The final recipe used 6,000 updates, AdamW, cosine learning-rate decay with a high floor of 0.99, and exponential moving average (EMA) weights with decay 0.999. SwiGLU and RMSNorm were tested as ablations. SwiGLU helped under a shorter training recipe but was worse under the final long recipe, while RMSNorm did not provide a stable gain. The final model therefore used GELU and LayerNorm.

The frozen final checkpoint obtained validation BPB **1.573665** and test BPB **1.593583** under CPU FP32 evaluation. The supplied baseline obtained test BPB **2.101260**, so the final model reduced test BPB by **0.507678** (24.16%). Local CPU scoring took 33.80 s compared with 10.49 s for the local baseline, or 3.22 times the baseline, below the 5-times limit. The measured peak working set was 1.5677 GiB and the listed uncompressed inference assets were 17.7179 MiB, both within the assignment limits.

1. Task and evaluation protocol

The assignment asks for a language model trained from scratch and evaluated on the supplied WikiText-2 benchmark. The ranking metric is bits per byte (BPB), with lower values being better. The supplied evaluator uses the fixed train-fitted BPE-2048 tokenizer and scores independent causal windows of 256 targets. Each prediction can use only the observed prefix in its own window; no state is carried across windows.

The evaluation protocol is `7506-mp1-wt2-v2`. The validation split contains 376,599 scored targets and 1,148,007 UTF-8 bytes. The test split contains 428,405 scored targets and 1,292,013 UTF-8 bytes. BPB is computed from the summed negative log probability in base 2 divided by the complete raw UTF-8 byte length. Token perplexity is recorded for reference but is not the ranking metric.

The assignment imposes three test-time limits for one frozen predictor:

1. CPU scoring time must be no more than five times the baseline measured on the same machine.
2. Peak evaluation RAM must be no more than 4 GiB.
3. Uncompressed inference assets must be no more than 64 MiB.

The supplied data, tokenizer, evaluator, and causal-window protocol were kept unchanged. All model and training decisions were made using validation results. The test result reported here was produced after freezing the final method and was not used for model selection.

2. Environment and implementation

Experiments were run on a laptop with an AMD Ryzen 9 7945HX CPU and an NVIDIA GeForce RTX 4060 Laptop GPU with 8 GiB of VRAM. Training used CUDA BF16 in the logged final runs and four CPU threads for data and evaluation utilities. The recorded software environment for the final run was Python 3.12 with PyTorch 2.7.1+cu126. Ranked evaluation was performed with CPU FP32, as required by the assignment.

The supplied baseline is `model.py` with `configs/baseline.json`: width 128, four heads, four blocks, context 256, and 1,088,256 parameters. The final `student.py` model uses width 224, seven heads, six blocks, context 256, and 4,146,688 parameters. The configuration file `configs/student_gelu_w224h7d6.json` selects the GELU branch. The implementation also retains a parameter-matched SwiGLU branch for the ablation experiments. Each Transformer block uses pre-normalization with two LayerNorm modules, and the final output uses LayerNorm.

The training recipe was implemented through the supplied `train.py`. It uses AdamW with base learning rate 0.001 and weight decay 0.1, gradient clipping at 1.0, a 100-update warm-up, and cosine scheduling. For a floor f , the learning rate after warm-up is

$$\text{lr}(t) = 0.001 \times [f + (1 - f) \times (1 + \cos(\pi t/T)) / 2]$$

Here T is the total number of updates. The final run used $f = 0.99$, so the learning rate decayed only slightly from the base value. EMA weights were used for validation and saved in the final checkpoint; the final decay was 0.999.

3. Experimental design

The experiments were controlled comparisons on the same data and tokenizer; unless a table states otherwise, validation used FP32 and a fixed seed. I varied capacity, training length, learning-rate floor, EMA, MLP activation, normalization, and seed sensitivity.

The supplied baseline used 1,200 updates; capacity trials increased width and/or depth at 3,600 updates, followed by 224/7/6 trials of the remaining recipe choices. The archive contains 45 logged `metrics.json` files, representing approximately 10,422 s (2.90 h) of logged process time; this excludes runs not written to the archive.

The final method was selected from validation results. The final test checkpoint was then frozen. Test BPB was not used to choose the architecture, training length, learning-rate floor, EMA decay, or seed.

4. Results

4.1 Baseline and capacity

The official baseline test result was 2.101260 BPB. A same-family validation comparison shows that increasing capacity substantially improved prediction quality under a common 3,600-update budget.

Implementation/configuration	Width	Heads	Depth	Parameters	Updates	Validation BPB
Official baseline (<code>model</code>)	128	4	4	1,088,256	1,200	2.071084

Implementation/configuration	Width	Heads	Depth	Parameters	Updates	Validation BPB
Larger baseline (model)	160	4	6	2,224,640	3,600	1.726376
Larger baseline (model)	192	6	8	4,001,664	3,600	1.675132
Larger baseline (model)	224	7	6	4,146,688	3,600	1.673092

The 224/7/6 configuration gave the best validation BPB among the capacity trials while remaining small enough for the CPU evaluation limits. This motivated using it for the subsequent training-recipe and architecture ablations.

4.2 Learning-rate floor

The learning-rate floor was scanned with the 224/7/6 model, SwiGLU, EMA 0.999, 4,800 updates, and seed 17. The result improved monotonically over the tested range:

Learning-rate floor	Validation BPB
0.05	1.633915
0.10	1.629174
0.20	1.626683
0.30	1.624230
0.40	1.618411
0.50	1.614618
0.80	1.604672
0.99	1.600906

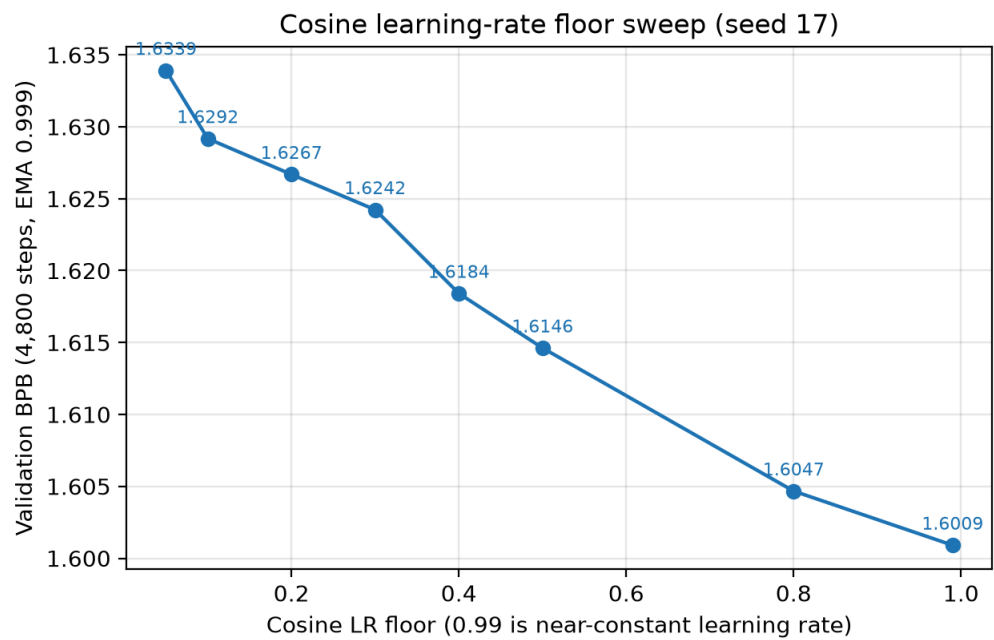


Figure 1. Validation BPB decreases across the eight tested cosine learning-rate floors. The 0.99 setting is close to a constant learning rate, but it is still implemented by the cosine schedule.

This result does not mean that a floor of 0.99 is universally optimal. It means that, for this model, optimizer, EMA setting, and training budget, a high final learning rate was preferable within the tested range. The final recipe therefore used 0.99, while the possibility of an optimum above the tested range remains unexamined.

4.3 Training length and the validation turning point

The final GELU student run and its 7,200-update control gave the following validation checkpoints:

Update	Final GELU student	7,200-step control
3,600	1.614944	1.614989
4,800	1.579509	1.579358
6,000	1.573665	1.573882
7,200	-	1.574851

The control used the same 224/7/6 scale, EMA 0.999, learning-rate floor 0.99, and seed 18.

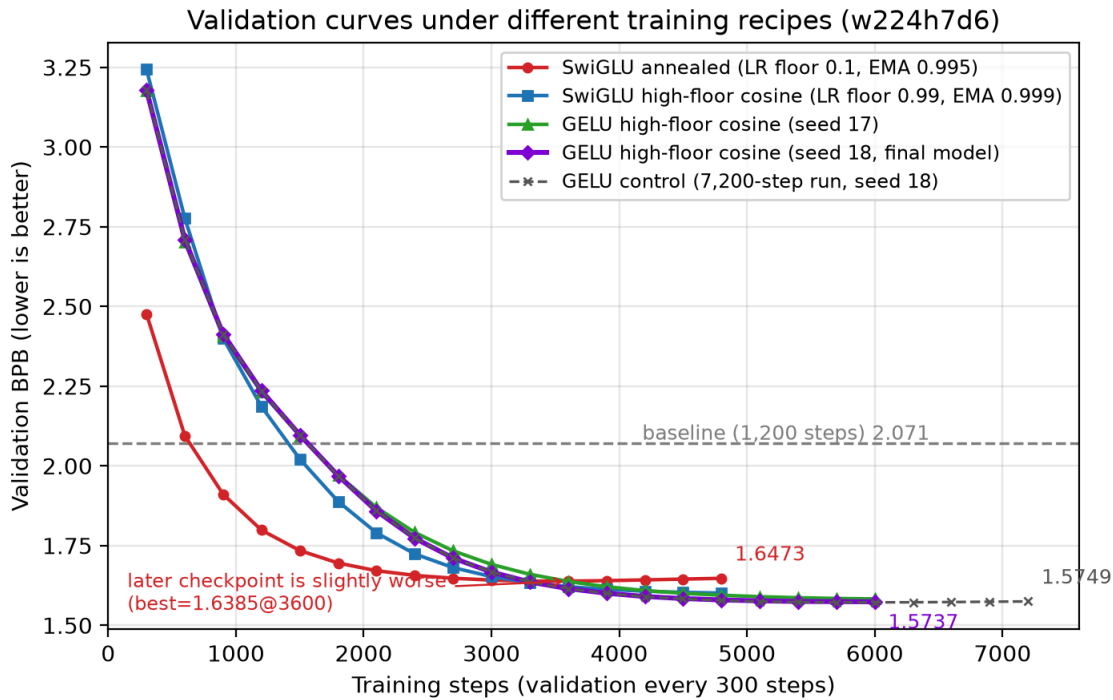


Figure 2. Validation trajectories for the main training recipes. The dashed grey curve is the 7,200-step GELU control trajectory; it is included to show the small late validation increase and is not the submitted checkpoint.

The curve improved through 6,000 updates and then rose slightly at 7,200. This is a small validation turning point rather than evidence of severe overfitting. It supports selecting 6,000 updates as a good stopping point under the tested recipe. The 7,200-point curve is a control trajectory used to study training length; the frozen final checkpoint is the separately recorded GELU student checkpoint.

4.4 MLP ablation

SwiGLU was tested because its gated feed-forward structure can provide a useful nonlinear interaction at approximately matched MLP parameter count. Its effect depended on the training recipe. Under the shorter 3,600-update recipe with EMA 0.995 and learning-rate floor 0, SwiGLU improved validation BPB from 1.673092 for the GELU control to 1.658759. Under the final long recipe, however, GELU was better.

Model/activation	Seed	Updates	EMA	LR floor	Validation BPB
GELU control (mode1)	17	6,000	0.999	0.99	1.582604
GELU student	17	6,000	0.999	0.99	1.582478
SwiGLU student	17	6,000	0.999	0.99	1.596944
GELU student	18	6,000	0.999	0.99	1.573665
SwiGLU student	18	6,000	0.999	0.99	1.597318

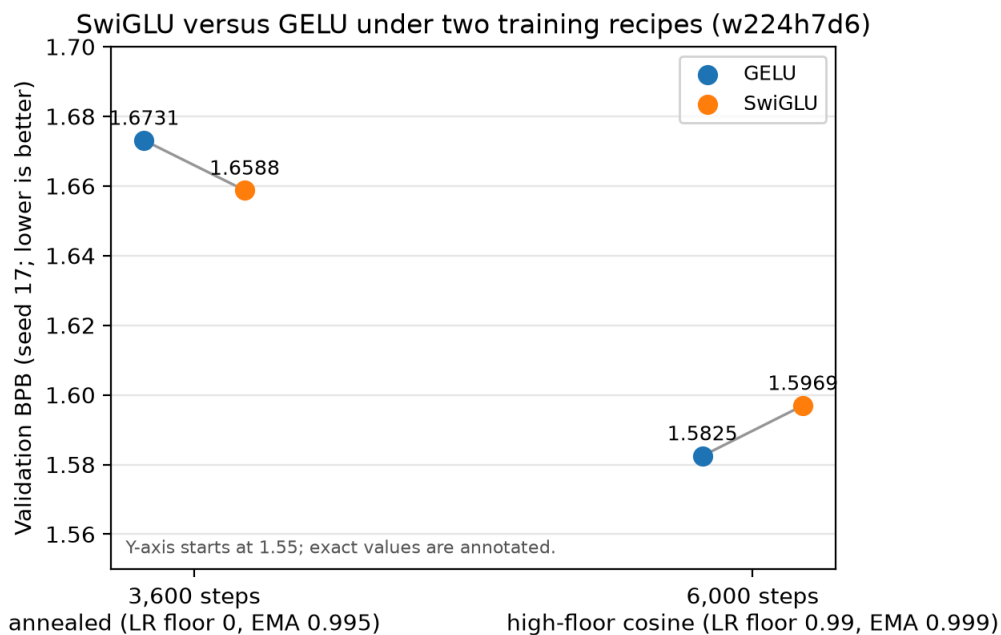


Figure 3. SwiGLU and GELU under two different training recipes. The plot is a recipe-dependent ablation, not a pure training-duration-only comparison, because the learning-rate floor and EMA decay also change between the two groups.

The correct conclusion is therefore conditional: SwiGLU was useful in one shorter-training regime, but it was not the best choice after the training recipe was changed to the final 6,000-update, high-floor, high-EMA setting. This interaction is why the report treats the MLP result as an ablation rather than claiming that one activation is always superior.

4.5 RMSNorm ablation

RMSNorm was tested by replacing only the two LayerNorm modules inside each Transformer block. The final LayerNorm was kept unchanged. The results were mixed and did not show a reproducible advantage:

Recipe	Seed	LayerNorm + SwiGLU	RMSNorm + SwiGLU	Difference (RMS – LN)
3,600 steps, EMA 0.995, LR floor 0.10	17	1.652313	1.652011	–0.000302
6,000 steps, EMA 0.999, LR floor 0.99	17	1.596944	1.597629	+0.000685
6,000 steps, EMA 0.999, LR floor 0.99	18	1.597318	1.597260	–0.000058

Because the sign and magnitude of the difference changed with the recipe and seed, the final model retained LayerNorm. The experiment was useful as a negative result: it prevented a single small improvement in one run from being presented as a general normalization benefit.

4.6 Seed sensitivity

The final recipe was checked across seeds. For the larger GELU control, the recorded validation BPB values were 1.581154 (seed 16), 1.582604 (seed 17), and 1.573882 (seed 18). The final student seed-18 run reached 1.573665, while the student seed-17 run reached 1.582478. The difference between seeds is large enough that a single lucky seed should be treated cautiously.

The final test score is therefore reported together with the seed and full checkpoint hash. Future work should repeat the final recipe over more seeds or select a checkpoint using a predeclared multi-seed rule. This was not done because the assignment deadline required freezing the current reproducible run.

Data note. The seed-16 and seed-18 comparisons in the run archive are available under their corresponding `runs/` directories. The report does not infer a population mean from these few seeds; it only uses them to expose run-to-run variability.

5. Final frozen evaluation

The final checkpoint is:

`outcome/gelu-w224h7d6-6000-cuda-ema999-1rf099-18/checkpoint.pt`

Its final configuration and training record are:

Field	Value
Implementation	<code>student</code>
Configuration	<code>student_gelu_w224h7d6.json</code>
Width / heads / depth	224 / 7 / 6
MLP / normalization	GELU / LayerNorm
Parameters	4,146,688
Updates / processed targets	6,000 / 49,152,000

Field	Value
Seed	18
Training precision	BF16 on CUDA
EMA decay	0.999
Learning-rate floor	0.99

The frozen CPU FP32 validation and test results are:

Split	BPB	Token perplexity	Targets	UTF-8 bytes	Scoring device
Validation	1.573665	27.8014	376,599	1,148,007	CPU FP32
Test	1.593583	27.9744	428,405	1,292,013	CPU FP32

Compared with the official test baseline of 2.101260 BPB, the final test result is 0.507678 BPB lower, a 24.16% relative reduction. The test score is higher than validation by approximately 0.019918 BPB, which is consistent with selecting the method on validation and then evaluating a held-out split.

5.1 Timing limit

The README reports 5.92 s as a reference-machine baseline scoring time. On this laptop, the local baseline JSON records 10.4903 s, which is 1.77 times the README reference. This is a hardware difference, not a separate assignment allowance.

The final test JSON records 33.8021 s on the same local CPU. The directly comparable ratio is $33.8021 / 10.4903 = 3.22$ times, which is below the allowed 5× local-baseline limit. The final model is slower than the baseline because it has approximately 3.81 times as many parameters, but it remains within the scoring budget.

5.2 RAM and inference assets

The resource measurement was run with `measure_resources.ps1` against the frozen checkpoint and CPU FP32 evaluator. The recorded values were:

Resource	Measured	Limit	Status
Peak working set	1.5677 GiB	4 GiB	Pass
Peak private bytes sampled	3.0340 GiB	4 GiB	Informational; also below limit
Listed uncompressed inference assets	17.7179 MiB	64 MiB	Pass

The listed assets include the checkpoint, `student.py`, `common.py`, `evaluate.py`, the final configuration, the data manifest, and the tokenizer. The measurement is conservative with respect to the explicitly listed files; the complete `code` directory including the checkpoint was also below 64 MiB in the local check.

6. Critical analysis and limitations

The improvement cannot be attributed to one isolated architectural change. The final result combines a larger model, a longer training budget, a high learning-rate floor, EMA, and a selected seed. The capacity table shows that model size alone produced a large improvement over the small baseline. The learning-rate scan shows that the schedule mattered. The MLP and RMSNorm ablations show that attractive architectural substitutions were not uniformly beneficial under the final recipe.

The validation search also creates a selection bias: many candidate settings were compared on the same validation split, so the reported validation score is an optimistic estimate of the search procedure's generalization. The held-out test score was reserved for the frozen final method, but the final seed was selected after substantial validation experimentation. More independent seeds would provide a stronger estimate of robustness.

The 7,200-step turning point is small, so it should be interpreted as a practical stopping signal rather than a definitive proof of overfitting. Likewise, the RMSNorm differences are close to the noise scale suggested by the seed variation. The report therefore emphasizes the reproducible final checkpoint and its resource measurements rather than claiming universal superiority for any one component.

Training used CUDA BF16 for speed, but all ranking numbers in this report came from CPU FP32 evaluation. Training time and search time are not the leaderboard metric; they are disclosed to make the experimental cost transparent.

7. Reproduction instructions

All commands below are run from the extracted project's `code/` directory. In PowerShell, first move to the project root and then enter `code/`:

7.1 Install and contract checks

```
cd .\code
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install torch==2.7.1 --index-url https://download.pytorch.org/whl/cu126
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
```

For CPU-only installation, replace the CUDA PyTorch index with the CPU index documented in `README.md`. The evaluator command below remains CPU FP32 in either environment.

7.2 Reproduce the final training run

```
python train.py `
  --implementation student `
  --config configs/student_gelu_w224h7d6.json `
  --device cuda `
  --precision bf16 `
  --threads 4 `
  --seed 18 `
  --steps 6000 `
  --batch-size 32 `
  --eval-every 300 `
  --lr-floor 0.99 `
  --ema-decay 0.999 `
  --run-dir outcome/reproduce
```

The run writes `checkpoint.pt` and `metrics.json`. Training is not needed to evaluate the submitted checkpoint; it is included to document how the checkpoint was produced.

7.3 Reproduce validation and test scores

```
python evaluate.py `
  --checkpoint outcome/gelu-w224h7d6-6000-cuda-ema999-lrf099-18/checkpoint.pt `
  --device cpu `
  --precision fp32 `
  --threads 4 `
  --split validation `
  --output outcome/reproduce/reproduced-validation.json

python evaluate.py `
  --checkpoint outcome/gelu-w224h7d6-6000-cuda-ema999-lrf099-18/checkpoint.pt `
  --device cpu `
  --precision fp32 `
  --threads 4 `
  --split test `
  --output outcome/reproduce/reproduced-test.json
```

The expected test BPB is approximately 1.593583. Small timing differences are expected across machines; the evaluator protocol, precision, split, and checkpoint must remain unchanged.

7.4 Reproduce resource measurements

```
PowerShell -ExecutionPolicy Bypass -File .\measure_resources.ps1 `
  -Checkpoint ./outcome/gelu-w224h7d6-6000-cuda-ema999-lrf099-18/checkpoint.pt `
  -Output ./outcome/reproduced-resource-measurement.json `
  -Threads 4 `
  -AssetPath
./student.py,./common.py,./evaluate.py,./configs/student_gelu_w224h7d6.json,./data/manifest.
json,./data/tokenizer.json
```

The resource script invokes CPU FP32 test evaluation, samples the evaluator process, records peak working set and private bytes, and sums the explicitly listed uncompressed assets. It does not modify the checkpoint or the evaluator.

8. Compliance and artifact audit

The following checklist records how the final artifact satisfies the published rules.

Requirement	Evidence or statement
Training data	Only the supplied WikiText-2 training split was used for optimization.
Validation selection	Architecture, training recipe, checkpoint length, and seed comparisons used validation results.
Test isolation	The test split was reserved for the frozen final evaluation and was not used to tune the method. Any post-freeze repeat was for timing/resource or reproduction checks only.
Data/tokenizer/evaluator	Supplied data, <code>data/tokenizer.json</code> , <code>common.py</code> , and <code>evaluate.py</code> were retained unchanged for the reported score.
Causality	The supplied causal-window evaluator and the contract tests were used; no cross-window state or future-token access was introduced.
CPU FP32 ranking score	Test evaluation used <code>--device cpu --precision fp32</code> .
Timing limit	33.8021 s final / 10.4903 s local baseline = 3.22×, below 5×.
RAM limit	Peak working set 1.5677 GiB, below 4 GiB.
Asset limit	Listed assets 17.7179 MiB, below 64 MiB.
Checkpoint hash	<code>8f6b9acb10861085543724ab4423f31b2aad12dc538f64b89e0a68c8042320e</code>
Implementation hash	<code>b4c92c8d0d87a9e4558819102abf6c398d04e4cfae79fce5ddfbbdb8a7453a9e4</code>
Evaluator hash	<code>128bcb2dab0be0d427505bddb4671e3ab3a8f78e114be79a689c0f9029af133d</code>
Tokenizer hash	<code>020d1bc6aa4449c4f352b2e03d0e0fb4f39287f15297705e421b1fa7d817262e</code>

The hash values above are copied from the final evaluation JSON and identify the exact predictor used for the reported result. Before submission, the release bundle should be checked once more so that any manifest file included in the bundle agrees with the final implementation hash.

References

1. DASE7506 MP1 assignment guide, `GUIDE.md`.
2. DASE7506 MP1 code README, `code/README.md`.
3. Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher, “Pointer Sentinel Mixture Models,” 2016, the reference work introducing WikiText-2.
4. WikiText-2 raw data and the accompanying CC BY-SA 3.0 / GNU Free Documentation License notices documented in `README.md`.